Name: Akansha Dilip Ghoderao

Roll No: 09

Assignment No: 1

Title: Download Heart dataset

a) Find shape of data
b) Find missing values
c) Find data type of each column
d) Finding out zero's
e) Fine mean age of patients
f) Now extract only age, sex, Chestpain, RestBP, Chol, randomly divide the dataset in training (75%) testing (25%)

A) Read Dataset

[1]:
```
import pandas as pd
df=pd.read_csv("Heart.csv")
```

[2]:
```
df
```

[2]:

	Unnamed: 0	Age	Sex	ChestPain	RestBP	Chol	Fbs	RestECG	MaxHR	ExAng	Oldpeak	Slope	Ca	Thal	AHD
0	1	63	1	typical	145	233	1	2	150	0	2.3	3	0.0	fixed	No
1	2	67	1	asymptomatic	160	286	0	2	108	1	1.5	2	3.0	normal	Yes
2	3	67	1	asymptomatic	120	229	0	2	129	1	2.6	2	2.0	reversable	Yes
3	4	37	1	nonanginal	130	250	0	0	187	0	3.5	3	0.0	normal	No
4	5	41	0	nontypical	130	204	0	2	172	0	1.4	1	0.0	normal	No
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
298	299	45	1	typical	110	264	0	0	132	0	1.2	2	0.0	reversable	Yes
299	300	68	1	asymptomatic	144	193	1	0	141	0	3.4	2	2.0	reversable	Yes
300	301	57	1	asymptomatic	130	131	0	0	115	1	1.2	2	1.0	reversable	Yes
301	302	57	0	nontypical	130	236	0	2	174	0	0.0	2	1.0	normal	Yes
302	303	38	1	nonanginal	138	175	0	0	173	0	0.0	1	NaN	normal	No

303 rows × 15 columns

B) Find shape of data & c) Find data type of each column

```
[3]: print("\n Data Type\n",df.shape)
```

```
 Data Type
 (303, 15)
```

```
[4]: print("\n Dataset Data Types/n",df.dtypes)
```

```
 Dataset Data Types/n Unnamed: 0      int64
Age                  int64
Sex                  int64
ChestPain           object
RestBP               int64
Chol                 int64
Fbs                  int64
RestECG              int64
MaxHR                int64
ExAng                int64
Oldpeak            float64
Slope                int64
Ca                 float64
Thal                object
AHD                 object
dtype: object
```

d) Find missing values & e) Find out zero's

```
[5]: print("\n Missing values per columns\n",df.isna().sum())
```

```
 Missing values per columns
 Unnamed: 0    0
Age            0
Sex            0
ChestPain      0
RestBP         0
Chol           0
Fbs            0
RestECG        0
MaxHR          0
ExAng          0
Oldpeak        0
Slope          0
Ca             4
Thal           2
AHD            0
dtype: int64
```

```
[6]: print("\n Zero counts per column\n",(df==0).sum())
```

```
 Zero counts per column
 Unnamed: 0      0
Age              0
Sex             97
ChestPain        0
RestBP           0
Chol             0
Fbs            258
RestECG        151
MaxHR            0
ExAng          204
Oldpeak         99
Slope            0
Ca             176
Thal             0
AHD              0
dtype: int64
```

f) Find mean of age of patients & g) Now extract only age, sex, chestpain, RestBP, chol, randomly, divide the dataset in training (75%) and testing (25%)

[7]:
```
mean=df['Age'].mean()
print("Mean Age",mean)
```

```
Mean Age 54.43894389438944
```

[8]:
```
import sklearn
from sklearn.model_selection import train_test_split
cols=['Age','Sex','ChestPain','RestBP','Chol']
df_subset=df[cols]
train_df, test_df=train_test_split(df_subset, test_size=0.25, random_state=42)
print(f"Training set Shape: {train_df.shape}")
print(f"Testing set Shape: {test_df.shape}")
```

```
Training set Shape: (227, 5)
Testing set Shape: (76, 5)
```

Name: Akansha Dilip Ghoderao

Roll No: 09

Assignment No: 2

Title: Given a set of sample points in N dimensional feature space. Write a program to fit the points with a hyper plane using Linear Regression. Calculate sum of residual error.

Source Code

[8]:
```
import numpy as np
from sklearn.linear_model import LinearRegression
```

[9]:
```
np.random.seed(52)
X=np.random.rand(10,3)*10
y=5+2*X[:,0]+3*X[:,1]-X[:,2]+np.random.randn(10)
model=LinearRegression()
model.fit(X,y)
y_pred=model.predict(X)
residuals=y-y_pred
sum_residual_error=np.sum(residuals)
print("Coefficients Weight:",model.coef_)
```

Output

```
Coefficients Weight: [ 2.0063521   2.87411937 -0.85027946]
```

```
print("Intercept (bias term):",model.intercept_)
```

Output

```
Intercept (bias term): 4.799954496188544
```

```
print("Predictions:",y_pred)
```

Output

```
Predictions: [20.2729686  14.75963637 25.1551369  27.04834184 25.61543969 34.10991307
 33.64328895 12.51320467 36.88991401 40.26512294]
```

```
print("Residuals:",residuals)
```

Output

```
Residuals: [ 1.31061126 -0.59216281  1.56667491 -0.54275821 -0.49806156 -0.50860118
 -0.86723662 -0.42039024  0.07076965  0.48115481]
```

```
print("Sum of Residual Error:",sum_residual_error)
```

Output

```
Sum of Residual Error: 3.552713678800501e-15
```

Name: Akansha Dilip Ghoderao

Roll No: 09

Assignment No: 3

Title: Write a program that provides option to compute different distance measures between two points in the N dimensional feature space. Consider some sample datasets for computing distances among sample points.

[1]:
```
import numpy as np
```

[2]:
```
from scipy.spatial import distance

def compute_distance(point1, point2, method, p=2):
    if method == "euclidean":
        return distance.euclidean(point1, point2)
    elif method == "manhattan":
        return distance.cityblock(point1, point2)
    elif method == "minkowski":
        return distance.minkowski(point1, point2, p)
    elif method == "chebyshev":
        return distance.chebyshev(point1, point2)
    elif method == "cosine":
        return distance.cosine(point1, point2)
    else:
        raise ValueError("Unknown method selected!")
```

[3]:
```
dataset=np.array([
            [3,2,1],
            [5,6,4],
            [1,0,1],
            [4,4,4]])
print("Sample Dataset (each row is point in N-dimensional space):")
```

Output

```
Sample Dataset (each row is point in N-dimensional space):
```

```
print(dataset)
```

Output

```
[[3 2 1]
 [5 6 4]
 [1 0 1]
 [4 4 4]]
```

```
p1=dataset[0]
p2=dataset[1]
print("\nComputing distances between points:",p1,"and",p2)
```

Output

```
Computing distances between points: [3 2 1] and [5 6 4]
```

```
print("Euclidean distance:", compute_distance(p1,p2, "euclidean"))
```

Output

```
Euclidean distance: 5.385164807134504
```

```
print("Manhattan distance:", compute_distance(p1,p2,"manhattan"))
```

Output

```
Manhattan distance: 9
```

```
print("Cosine distance:", compute_distance(p1, p2, "cosine"))
```

Output

```
Cosine distance: 0.05582539899658234
```

```
print("Minkowski (p=3):", compute_distance(p1, p2, "minkowski", p=3))
```

Output

```
Minkowski (p=3): 4.626065009182741
```

```
print("Chebyshev distance:",compute_distance(p1,p2,"chebyshev"))
```

Output

```
Chebyshev distance: 4
```

Name: Akansha Dilip Ghoderao

Roll No: 09

Assignment No: 4

Title: Develop a program to Compute the correlation matrix to understand the relationships between pairs of features. Visualize the correlation matrix using a heatmap to know which variables have strong positive/negative correlations. Create a pair plot to visualize pairwise relationships between features. Use California Housing dataset.

Source Code

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing

housing = fetch_california_housing(as_frame=True)
df = housing.frame
print("Sample Data:")
print(df.head())
```

Output:

```
Sample Data:
   MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  Latitude  \
0  8.3252      41.0  6.984127   1.023810       322.0  2.555556     37.88
1  8.3014      21.0  6.238137   0.971880      2401.0  2.109842     37.86
2  7.2574      52.0  8.288136   1.073446       496.0  2.802260     37.85
3  5.6431      52.0  5.817352   1.073059       558.0  2.547945     37.85
4  3.8462      52.0  6.281853   1.081081       565.0  2.181467     37.85

   Longitude  MedHouseVal
0    -122.23        4.526
1    -122.22        3.585
2    -122.24        3.521
3    -122.25        3.413
4    -122.25        3.422
```

Code

```
corr_matrix = df.corr()
print("\nCorrelation Matrix:")
print(corr_matrix)
```

Output:

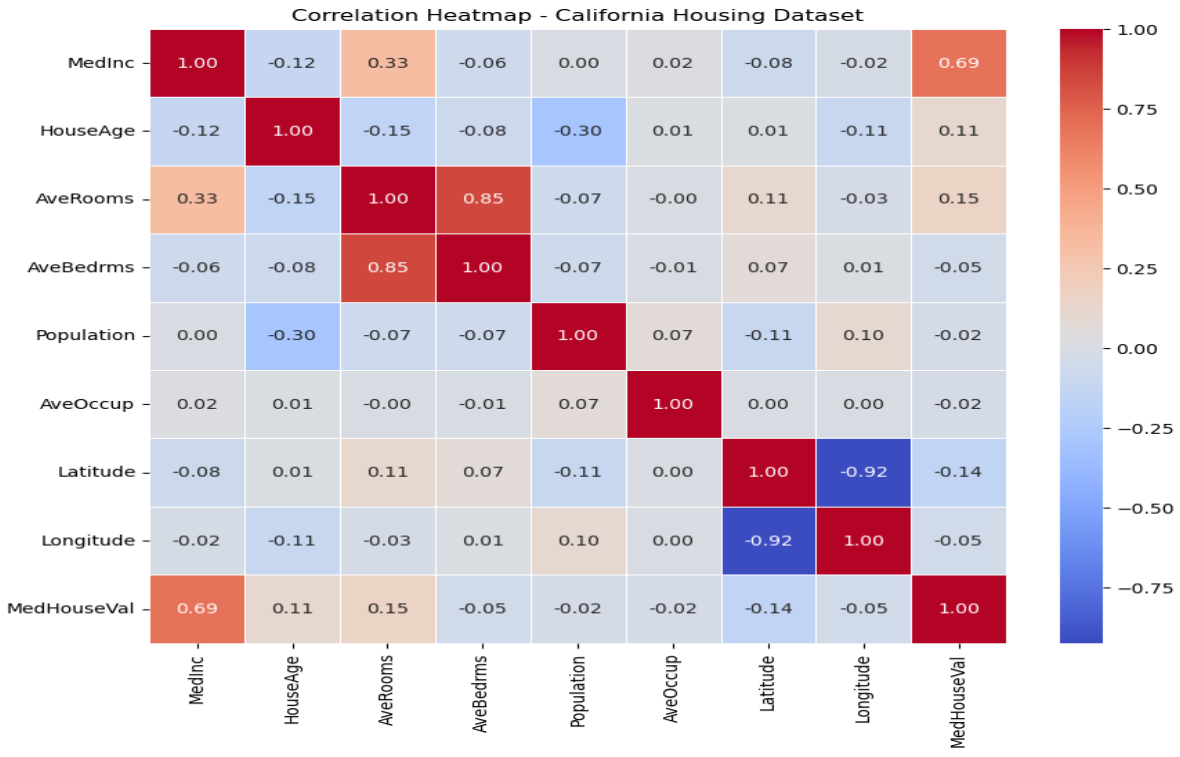
```
Correlation Matrix:
                MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  \
MedInc        1.000000 -0.119034  0.326895  -0.062040    0.004834  0.018766
HouseAge     -0.119034  1.000000 -0.153277  -0.077747   -0.296244  0.013191
AveRooms      0.326895 -0.153277  1.000000   0.847621   -0.072213 -0.004852
AveBedrms    -0.062040 -0.077747  0.847621   1.000000   -0.066197 -0.006181
Population    0.004834 -0.296244 -0.072213  -0.066197    1.000000  0.069863
AveOccup      0.018766  0.013191 -0.004852  -0.006181    0.069863  1.000000
Latitude     -0.079809  0.011173  0.106389   0.069721   -0.108785  0.002366
Longitude    -0.015176 -0.108197 -0.027540   0.013344    0.099773  0.002476
MedHouseVal   0.688075  0.105623  0.151948  -0.046701   -0.024650 -0.023737

              Latitude  Longitude  MedHouseVal
MedInc       -0.079809  -0.015176     0.688075
HouseAge      0.011173  -0.108197     0.105623
AveRooms      0.106389  -0.027540     0.151948
AveBedrms     0.069721   0.013344    -0.046701
Population   -0.108785   0.099773    -0.024650
AveOccup      0.002366   0.002476    -0.023737
Latitude      1.000000  -0.924664    -0.144160
Longitude    -0.924664   1.000000    -0.045967
MedHouseVal  -0.144160  -0.045967     1.000000
```

Code

```
plt.figure(figsize=(10,8))
sns.heatmap(corr_matrix, annot=True, cmap="coolwarm", fmt=".2f", linewidths=0.5)
plt.title("Correlation Heatmap - California Housing Dataset")
plt.show()
```

Output:

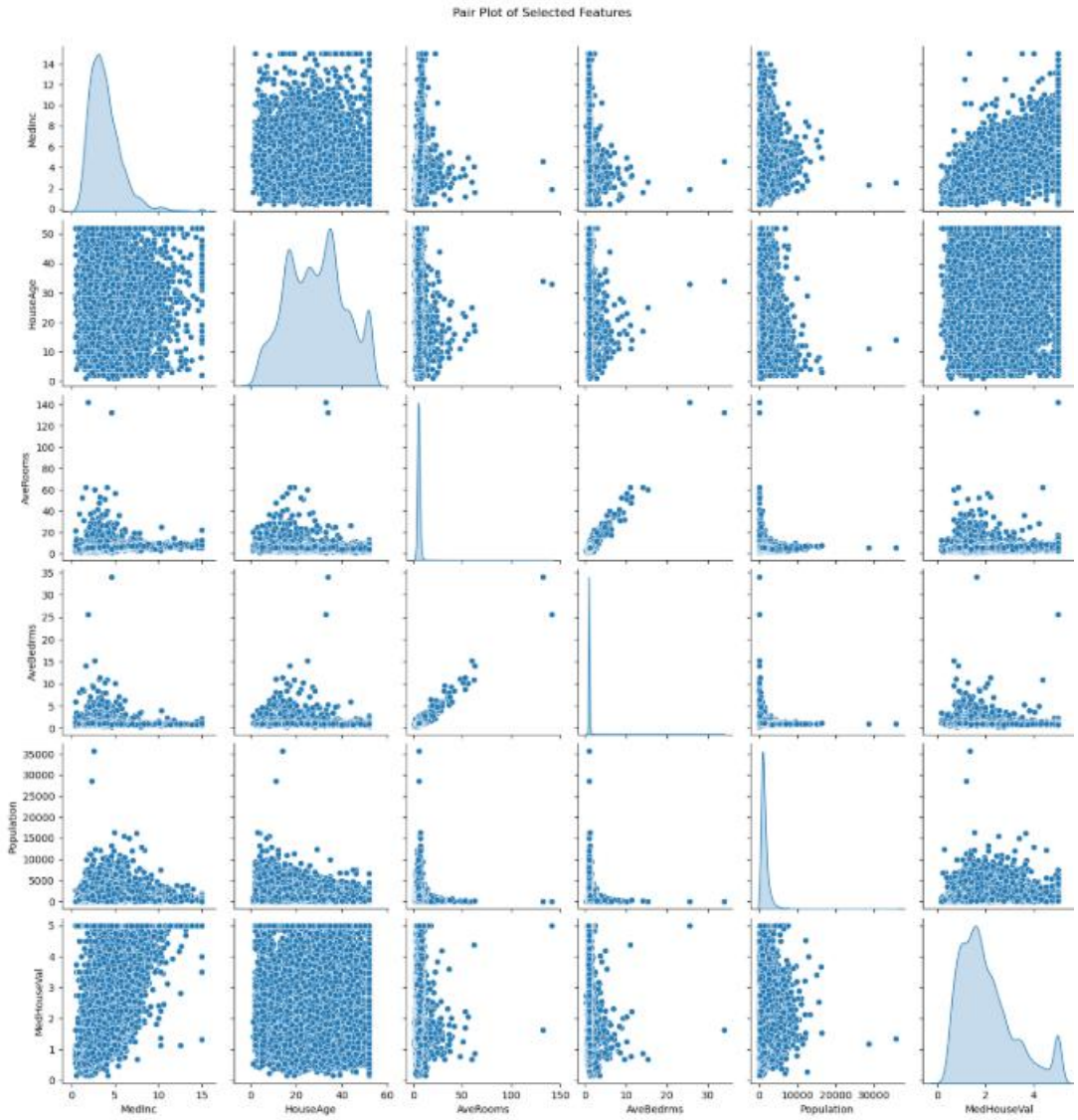


Code

```
subset_features = ["MedInc","HouseAge","AveRooms","AveBedrms","Population","MedHouseVal"]

sns.pairplot(df[subset_features], diag_kind="kde")
plt.suptitle("Pair Plot of Selected Features", y=1.02)
plt.show()
```

Output:



Name: Akansha Dilip Ghoderao
Roll No: 09

Assignment No. 5

Title: Write a program to implement the naïve Bayesian classifier for a sample training data set. Compute the accuracy of the classifier, considering few test data sets.

Source Code:

```
import math
from collections import defaultdict
```

```
dataset = [
    ['Sunny', 'Hot', 'High', 'False', 'No'],
    ['Sunny', 'Hot', 'High', 'True', 'No'],
    ['Overcast', 'Hot', 'High', 'False', 'Yes'],
    ['Rain', 'Mild', 'High', 'False', 'Yes'],
    ['Rain', 'Cool', 'Normal', 'False', 'Yes'],
    ['Rain', 'Cool', 'Normal', 'True', 'No'],
    ['Overcast', 'Cool', 'Normal', 'True', 'Yes'],
    ['Sunny', 'Mild', 'High', 'False', 'No'],
    ['Sunny', 'Cool', 'Normal', 'False', 'Yes'],
    ['Rain', 'Mild', 'Normal', 'False', 'Yes'],
    ['Sunny', 'Mild', 'Normal', 'True', 'Yes'],
    ['Overcast', 'Mild', 'High', 'True', 'Yes'],
    ['Overcast', 'Hot', 'Normal', 'False', 'Yes'],
    ['Rain', 'Mild', 'High', 'True', 'No']
]

train_data = dataset[:10]
test_data = dataset[10:]

class NaiveBayesClassifier:
    def __init__(self):
        self.class_probs = {}
        self.feature_probs = {}
```

```
    def fit(self, data):
        total = len(data)
        class_counts = defaultdict(int)
        feature_counts = defaultdict(lambda: defaultdict(lambda: defaultdict(int)))

        for row in data:
            *features, label = row
            class_counts[label] += 1
            for i, feat in enumerate(features):
                feature_counts[i][feat][label] += 1

        self.class_probs = {c: class_counts[c]/total for c in class_counts}

        self.feature_probs = {}
        for i in feature_counts:
            self.feature_probs[i] = {}
            for feat in feature_counts[i]:
                self.feature_probs[i][feat] = {}
                for c in class_counts:
                    # Laplace smoothing
                    self.feature_probs[i][feat][c] = (feature_counts[i][feat][c] + 1) / (class_counts[c] + len(feature_counts[i]))
```

```
    def predict(self, features):
        probs = {}
        for c in self.class_probs:
            prob = math.log(self.class_probs[c])  # log probabilities to avoid underflow
            for i, feat in enumerate(features):
                if feat in self.feature_probs[i]:
                    prob += math.log(self.feature_probs[i][feat].get(c, 1e-6))
                else:
                    prob += math.log(1e-6)  # very small probability if unseen
            probs[c] = prob
        return max(probs, key=probs.get)

    def score(self, data):
        correct = 0
        for row in data:
            *features, label = row
            if self.predict(features) == label:
                correct += 1
        return correct / len(data)

nb = NaiveBayesClassifier()
nb.fit(train_data)

print("Test Accuracy:", nb.score(test_data))
```

Output :

```
Test Accuracy: 0.75
```

Code :

```
print("\nPredictions:")
for row in test_data:
    *features, label = row
    print(f"Features: {features}, Predicted: {nb.predict(features)}, Actual: {label}")
```

Output :

```
Predictions:
Features: ['Sunny', 'Mild', 'Normal', 'True'], Predicted: No, Actual: Yes
Features: ['Overcast', 'Mild', 'High', 'True'], Predicted: Yes, Actual: Yes
Features: ['Overcast', 'Hot', 'Normal', 'False'], Predicted: Yes, Actual: Yes
Features: ['Rain', 'Mild', 'High', 'True'], Predicted: No, Actual: No
```

Name: Akansha Dilip Ghoderao

Roll No: 09

Assignment No: 6

Title: Write a program to implement k-Nearest Neighbour algorithm to classify the iris data set. Print both correct and wrong predictions using Python ML library classes.

Source Code

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

iris = load_iris()
X = iris.data
y = iris.target
target_names = iris.target_names

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=32)

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
```

Output:

```
KNeighborsClassifier
KNeighborsClassifier()
```

Code

```
y_pred = knn.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Output:

```
Accuracy: 1.0
```

Code

```
print("\nPrediction Results:")
for i in range(len(X_test)):
    actual = target_names[y_test[i]]
    predicted = target_names[y_pred[i]]
    if y_test[i] == y_pred[i]:
        print(f"Correct : Sample {i} - Predicted: {predicted}, Actual: {actual}")
    else:
        print(f"Wrong : Sample {i} - Predicted: {predicted}, Actual: {actual}")
```

Output:

```
Prediction Results:
Correct : Sample 0 - Predicted: versicolor, Actual: versicolor
Correct : Sample 1 - Predicted: setosa, Actual: setosa
Correct : Sample 2 - Predicted: setosa, Actual: setosa
Correct : Sample 3 - Predicted: versicolor, Actual: versicolor
Correct : Sample 4 - Predicted: virginica, Actual: virginica
Correct : Sample 5 - Predicted: virginica, Actual: virginica
Correct : Sample 6 - Predicted: setosa, Actual: setosa
Correct : Sample 7 - Predicted: setosa, Actual: setosa
Correct : Sample 8 - Predicted: versicolor, Actual: versicolor
Correct : Sample 9 - Predicted: setosa, Actual: setosa
Correct : Sample 10 - Predicted: versicolor, Actual: versicolor
Correct : Sample 11 - Predicted: virginica, Actual: virginica
Correct : Sample 12 - Predicted: versicolor, Actual: versicolor
Correct : Sample 13 - Predicted: versicolor, Actual: versicolor
Correct : Sample 14 - Predicted: virginica, Actual: virginica
Correct : Sample 15 - Predicted: virginica, Actual: virginica
Correct : Sample 16 - Predicted: versicolor, Actual: versicolor
Correct : Sample 17 - Predicted: virginica, Actual: virginica
Correct : Sample 18 - Predicted: versicolor, Actual: versicolor
Correct : Sample 19 - Predicted: setosa, Actual: setosa
Correct : Sample 20 - Predicted: setosa, Actual: setosa
Correct : Sample 21 - Predicted: virginica, Actual: virginica
Correct : Sample 22 - Predicted: virginica, Actual: virginica
Correct : Sample 23 - Predicted: setosa, Actual: setosa
Correct : Sample 24 - Predicted: setosa, Actual: setosa
Correct : Sample 25 - Predicted: versicolor, Actual: versicolor
Correct : Sample 26 - Predicted: setosa, Actual: setosa
Correct : Sample 27 - Predicted: virginica, Actual: virginica
Correct : Sample 28 - Predicted: setosa, Actual: setosa
Correct : Sample 29 - Predicted: setosa, Actual: setosa
Correct : Sample 30 - Predicted: versicolor, Actual: versicolor
Correct : Sample 31 - Predicted: setosa, Actual: setosa
Correct : Sample 32 - Predicted: setosa, Actual: setosa
Correct : Sample 33 - Predicted: virginica, Actual: virginica
Correct : Sample 34 - Predicted: versicolor, Actual: versicolor
Correct : Sample 35 - Predicted: setosa, Actual: setosa
Correct : Sample 36 - Predicted: setosa, Actual: setosa
Correct : Sample 37 - Predicted: virginica, Actual: virginica
Correct : Sample 38 - Predicted: virginica, Actual: virginica
Correct : Sample 39 - Predicted: versicolor, Actual: versicolor
Correct : Sample 40 - Predicted: setosa, Actual: setosa
Correct : Sample 41 - Predicted: virginica, Actual: virginica
Correct : Sample 42 - Predicted: setosa, Actual: setosa
Correct : Sample 43 - Predicted: virginica, Actual: virginica
Correct : Sample 44 - Predicted: setosa, Actual: setosa
```

Name: Akansha Dilip Ghoderao

Roll No: 09

Assignment No: 7

Title: Write a program to demonstrate the working of the decision tree based ID3 algorithm. Use an appropriate data set for building the decision tree and apply this knowledge to classify a new sample.

Source Code:

```
import math
from collections import Counter

dataset = [
    ['Sunny', 'Hot', 'High', 'False', 'No'],
    ['Sunny', 'Hot', 'High', 'True', 'No'],
    ['Overcast', 'Hot', 'High', 'False', 'Yes'],
    ['Rain', 'Mild', 'High', 'False', 'Yes'],
    ['Rain', 'Cool', 'Normal', 'False', 'Yes'],
    ['Rain', 'Cool', 'Normal', 'True', 'No'],
    ['Overcast', 'Cool', 'Normal', 'True', 'Yes'],
    ['Sunny', 'Mild', 'High', 'False', 'No'],
    ['Sunny', 'Cool', 'Normal', 'False', 'Yes'],
    ['Rain', 'Mild', 'Normal', 'False', 'Yes'],
    ['Sunny', 'Mild', 'Normal', 'True', 'Yes'],
    ['Overcast', 'Mild', 'High', 'True', 'Yes'],
    ['Overcast', 'Hot', 'Normal', 'False', 'Yes'],
    ['Rain', 'Mild', 'High', 'True', 'No']
]

features = ["Outlook", "Temperature", "Humidity", "Windy"]

def entropy(data):
    labels = [row[-1] for row in data]
    label_counts = Counter(labels)
    total = len(data)
    return -sum((count/total) * math.log2(count/total) for count in label_counts.values())

def information_gain(data, feature_index):
    base_entropy = entropy(data)
    values = set(row[feature_index] for row in data)
    new_entropy = 0
    for v in values:
        subset = [row for row in data if row[feature_index] == v]
        new_entropy += (len(subset)/len(data)) * entropy(subset)
    return base_entropy - new_entropy

def majority_class(data):
    labels = [row[-1] for row in data]
    return Counter(labels).most_common(1)[0][0]

def id3(data, feature_names):
    labels = [row[-1] for row in data]

    if labels.count(labels[0]) == len(labels):
        return labels[0]

    if not feature_names:
        return majority_class(data)

    gains = [information_gain(data, i) for i in range(len(feature_names))]
    best_index = gains.index(max(gains))
    best_feature = feature_names[best_index]

    tree = {best_feature: {}}
```

```
    values = set(row[best_index] for row in data)

    for v in values:
        subset = [row[:best_index] + row[best_index+1:] for row in data if row[best_index] == v]
        subtree = id3(subset, feature_names[:best_index] + feature_names[best_index+1:])
        tree[best_feature][v] = subtree

    return tree

def predict(tree, feature_names, sample):
    if isinstance(tree, str):
        return tree
    feature = next(iter(tree))
    feature_index = feature_names.index(feature)
    feature_value = sample[feature_index]

    if feature_value in tree[feature]:
        subtree = tree[feature][feature_value]
        return predict(subtree, feature_names, sample)
    else:
        return "Unknown"

tree = id3(dataset, features)
print("Decision Tree (ID3):")
print(tree)
```

Output:

```
Decision Tree (ID3):
{'Outlook': {'Overcast': 'Yes', 'Sunny': {'Humidity': {'Normal': 'Yes', 'High': 'No'}},
  'Rain': {'Windy': {'False': 'Yes', 'True': 'No'}}}}
```

Code:

```
sample = ['Sunny', 'Cool', 'High', 'True']  # New data
prediction = predict(tree, features, sample)
print("\nClassifying New Sample:", sample)
print("Predicted Class:", prediction)
```

Output:

```
Classifying New Sample: ['Sunny', 'Cool', 'High', 'True']
Predicted Class: No
```

Name: Akansha Dilip Ghoderao

Roll No: 09

Assignment No: 8

Title: Implement a Random Forest Classifier and evaluate its performance using the Wine dataset.

Source Code:

```
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

# Load Wine dataset
wine = load_wine()

X = wine.data
y = wine.target

# Split data into training and testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# Create Random Forest Classifier
model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

# Train the model
model.fit(X_train, y_train)

# Predict test data
y_pred = model.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Random Forest Classification")
print("----------------------------")
print("Accuracy:", accuracy)

# Display classification report
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# Predict a new wine sample
new_wine = [[14.23, 1.71, 2.43, 15.6, 127, 2.8, 3.06, 0.28, 2.29, 5.64, 1.04, 3.92, 1065]]

prediction = model.predict(new_wine)

print("Predicted Wine Class:", wine.target_names[prediction[0]])
```

Output:

```
Random Forest Classification
----------------------------
Accuracy: 1.0

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        19
           1       1.00      1.00      1.00        21
           2       1.00      1.00      1.00        14

    accuracy                           1.00        54
   macro avg       1.00      1.00      1.00        54
weighted avg       1.00      1.00      1.00        54

Predicted Wine Class: class_0
```

Name: Akansha Dilip Ghoderao

Roll No: 09

Assignment No: 9

Title: Write a program to implement K-means clustering algorithm. Select your own dataset to test the program. Demonstrate the nature of output with varying value of K.

Source Code:

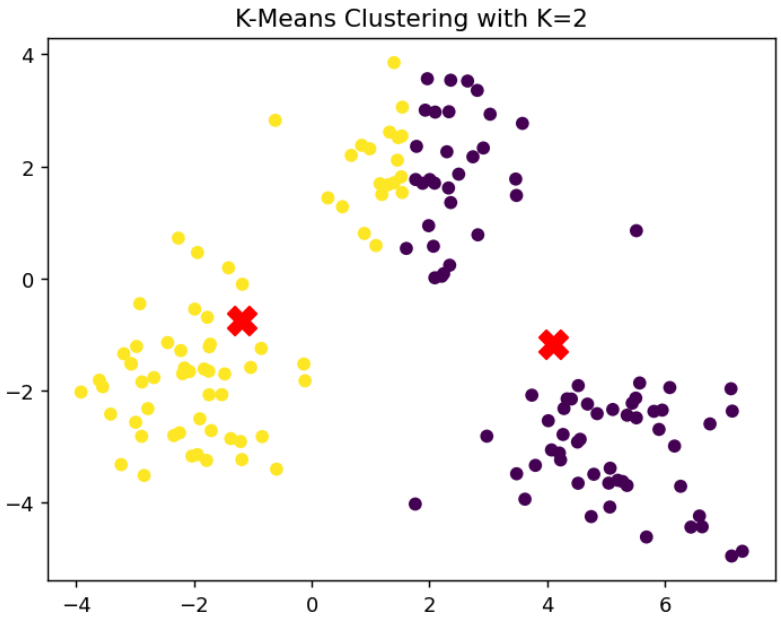
```
import numpy as np
import matplotlib.pyplot as plt

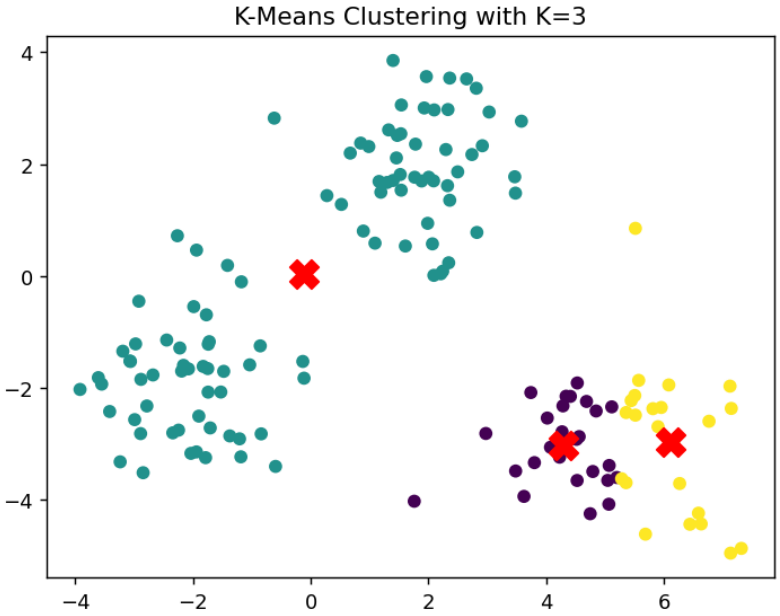

np.random.seed(42)
X1 = np.random.randn(50, 2) + np.array([2, 2])   # Cluster 1
X2 = np.random.randn(50, 2) + np.array([-2, -2]) # Cluster 2
X3 = np.random.randn(50, 2) + np.array([5, -3])  # Cluster 3
X = np.vstack((X1, X2, X3))

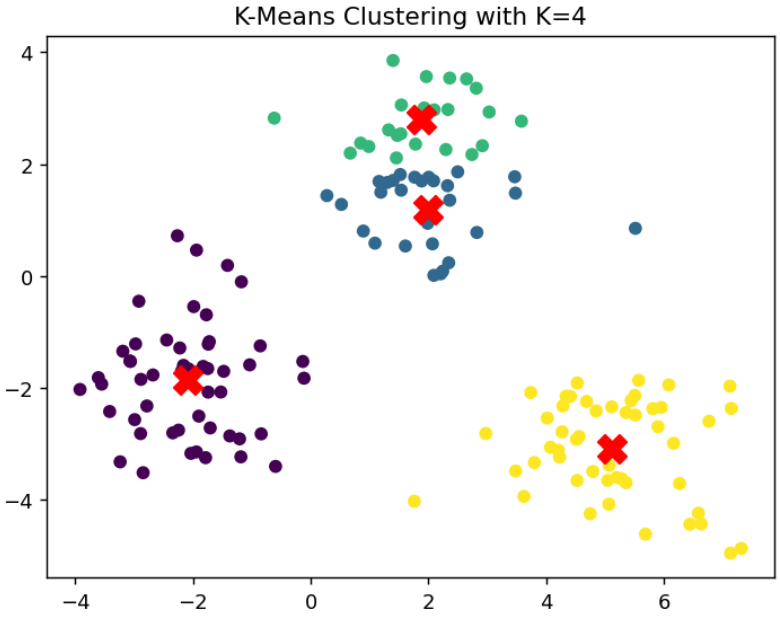

def kmeans(X, K, max_iters=100):
    centroids = X[np.random.choice(X.shape[0], K, replace=False)]
    for _ in range(max_iters):
        distances = np.linalg.norm(X[:, None] - centroids, axis=2)
        labels = np.argmin(distances, axis=1)
        new_centroids = np.array([X[labels==k].mean(axis=0) if len(X[labels==k])>0 else centroids[k]
                                  for k in range(K)])
        if np.allclose(centroids, new_centroids):
            break
        centroids = new_centroids
    return labels, centroids


for K in [2, 3, 4]:
    labels, centroids = kmeans(X, K)
    plt.scatter(X[:,0], X[:,1], c=labels, cmap="viridis", s=30)
    plt.scatter(centroids[:,0], centroids[:,1], c="red", marker="X", s=200)
    plt.title(f"K-Means Clustering with K={K}")
    plt.show()
```

Output:




K-Means Clustering with K=3
4
2
0
−2
−4
−4
−2
0
2
4
6


K-Means Clustering with K=4
4
2
0
−2
−4
−4
−2
0
2
4
6

Name: Akansha Dilip Ghoderao

Roll No: 09

Assignment No: 10

Title: Build an Artificial Neural Network by implementing the Backpropagation algorithm and test the same using appropriate data sets.

Source Code:

```
import numpy as np

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)

class NeuralNetwork:
    def __init__(self, x, y):
        self.input      = x
        self.weights1   = np.random.rand(self.input.shape[1], 4)
        self.weights2   = np.random.rand(4, 1)
        self.y          = y
        self.output     = np.zeros(self.y.shape)

    def feedforward(self):
        self.layer1 = sigmoid(np.dot(self.input, self.weights1))
        self.output = sigmoid(np.dot(self.layer1, self.weights2))

    def backprop(self):
        error = self.y - self.output
        d_output = error * sigmoid_derivative(self.output)
        error_hidden = d_output.dot(self.weights2.T)
        d_hidden = error_hidden * sigmoid_derivative(self.layer1)
        self.weights2 += self.layer1.T.dot(d_output)
        self.weights1 += self.input.T.dot(d_hidden)

    def train(self, epochs=10000):
        for _ in range(epochs):
            self.feedforward()
            self.backprop()

if __name__ == "__main__":
    X = np.array([[0,0],[0,1],[1,0],[1,1]])
    y = np.array([[0],[1],[1],[0]])
    nn = NeuralNetwork(X, y)
    nn.train(epochs=10000)
    print("Final Outputs after training:")
    print(nn.output.round(3))
```

Output:

```
Final Outputs after training:
[[0.017]
 [0.984]
 [0.985]
 [0.015]]
```